

UNIVERSITY OF THESSALY

SCHOOL OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

HOMEWORK 1

Optimization of Sobel Filter Application Code

Authors:

Christos Tzefronis (AEM 03629)
Christos Mountakis (AEM 03801)
Taxiarchis Papadinas (AEM 03865)

Instructor: Christos Antonopoulos

Department of Electrical & Computer Engineering
University of Thessaly

October 18, 2026

Contents

1 Execution Environment **2**

2 Baseline Implementation **2**

3 Optimizations **3**
- 3.1 Loop Interchange . . . 3
- 3.2 Function Inlining . . . 4
- 3.3 Loop Fusion . . . 5
- 3.4 Convolution Flatten . . . 6
- 3.5 Constant Folding . . . 7
- 3.6 Loop-Invariant Code Motion and Common Subexpression Elimination . . . 8
- 3.7 Strength Reduction . . . 9
- 3.8 Integer PSNR Accumulator . . . 10

4 Optimizations for `-O0` **12**
- 4.1 Loop Unrolling . . . 12
- 4.2 Memory Reads . . . 13
- 4.3 Square-Root Lookup Table . . . 14
- 4.4 PSNR Fusion . . . 15

5 Optimizations for `-O3 -ffast-math` **16**
- 5.1 Single-Precision Square Root . . . 16
- 5.2 Compiler Hints . . . 17

6 Conclusion **18**
- 6.1 Rejected Optimizations . . . 18
- 6.2 Final Results . . . 19

1 Execution Environment

All measurements were taken on the same machine, an Apple MacBook Pro with an M2 Pro processor, using the same compiler, Apple clang. Table 1 lists its configuration.

Table 1: System configuration

Component	Value
Hardware	
Machine	MacBook Pro (Mac14,9)
Processor	Apple M2 Pro (ARMv8, `arm64`)
Cores	10: 6 performance + 4 efficiency
L1 data cache	128 KB per performance core
L1 instruction cache	192 KB per performance core
Cache line (L1)	64 B
L2 cache	16 MB per performance cluster (3 cores)
RAM	16 GB LPDDR5
Software	
Operating system	macOS 27.0.1
Kernel	Darwin 27.0.0
Compiler	Apple clang 21.0.0
Compiler flags	`-Wall -O0` and `-Wall -O3 -ffast-math`

Clang does not provide the `-fast` flag, so the optimized series uses `-O3 -ffast-math`, which enables the same kind of aggressive optimizations, including the relaxed floating-point rules of `-fast`.

2 Baseline Implementation

The baseline (`00_sobel_orig`) is the unmodified, naive implementation of the Sobel filter with no manual optimizations applied. It serves as the reference point for all subsequent measurements: every speedup reported in this document is computed relative to the baseline within the same compiler flag set. The program was benchmarked using the same methodology throughout: 12 runs per configuration with the minimum and maximum discarded, reporting the mean and standard deviation of the remaining 10 runs.

Table 2: Baseline (`00_sobel_orig`) execution time

Flags	Mean (ms)	Std. dev. (ms)	Speedup
`-O0`	1282.5	0.8	1.00×
`-O3 -ffast-math`	173.9	0.2	1.00×

3 Optimizations

3.1 Loop Interchange

The baseline traverses the image in column-major order, resulting in a stride of 4096 bytes that fails to exploit spatial locality: each cache line serves only one pixel before it is evicted. Interchanging the loops to row-major order, both in the outer driver loop and within `convolution2D`, aligns memory accesses with C's row-major storage layout, reducing L1D load misses by ~98%, as confirmed by the hardware counter measurements in Table 5.

Table 3: Execution time at `-O0`: loop interchange

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`00_sobel_orig`	1282.5	0.8	1.00×
`01_sobel_loop_interchange`	1252.9	5.4	1.02×

Table 4: Execution time at `-O3 -ffast-math`: loop interchange

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`00_sobel_orig`	173.9	0.2	1.00×
`01_sobel_loop_interchange`	20.6	0.3	8.46×

(a) `-O0`

Metric	00	01	Diff.
L1D load misses	24.97 M	0.50 M	−98.0%
Cycles	4.51 G	4.39 G	−2.7%
IPC	3.74	3.85	+0.11

(b) `-O3 -ffast-math`

Metric	00	01	Diff.
L1D load misses	22.64 M	0.30 M	−98.7%
Cycles	614.1 M	71.4 M	−88.4%
IPC	1.16	4.09	+2.93

Table 5: Loop interchange: hardware counters

At `-O0` the baseline executes about 1000 instructions per output pixel (16.9 G instructions for 16.8 M pixels). Each L1D miss is served from the L2 cache: one column sweep touches one new 64 B line of `input` and one of `output` per row, i.e. $4096 \times 2 \times 64\,\text{B} \approx 512\,\text{KB}$, which is larger than the 128 KB L1D but fits easily in the 16 MB L2. The out-of-order core overlaps this latency with the surrounding independent instructions, so the misses are almost completely hidden: removing 98% of them saves only 2.7% of the cycles (1.02×).

At `-O3 -ffast-math` only about 42 instructions per pixel remain, too few to hide the miss latency, and the baseline becomes memory-bound (IPC 1.16). Removing the misses, and letting the compiler vectorize the now contiguous inner loop (16 pixels per iteration instead of 2), raises the IPC to 4.09 and gives 8.46×.

3.2 Function Inlining

In this step the `convolution2D` function was manually inlined into the main loop. The function is called twice per output pixel, i.e. $4094 \times 4094 \times 2 \approx 33.5\,\text{M}$ times, and every call carries a fixed overhead: the arguments are passed through the stack, a stack frame must be created on entry and released on return, and the call and return themselves are branches that redirect instruction fetch, adding a fixed latency penalty on every invocation.

Table 6: Execution time at `-O0`: function inlining

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`01_sobel_loop_interchange`	1252.9	5.4	1.02×
`02_sobel_function_inlining`	1124.0	2.2	1.14×

Table 7: Execution time at `-O3 -ffast-math`: function inlining

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`01_sobel_loop_interchange`	20.6	0.3	8.46×
`02_sobel_function_inlining`	20.6	0.1	8.46×

Table 8: Function inlining: hardware counters at `-O0`

Metric	01	02	Diff.
Instructions	16.87 G	16.80 G	−0.4%
Branches	2.799 G	2.732 G	−67.0 M
Cycles	4.41 G	3.95 G	−10.6%
IPC	3.82	4.26	+0.44

The branch count drops by exactly 67,043,344 = 33.5 M × 2, one call and one return per invocation (Table 8). The instruction count, however, drops by only 0.4%: at `-O0` the inlined code still loads its operands from the stack, so only about two instructions per call disappear. The gain comes from the cycles, which drop by 10.6%, about 14 cycles per call. The call sequence is expensive because of its latency: the arguments make a round trip through memory, and the call and return interrupt instruction fetch. Without them the same instructions run at a higher IPC (3.82 to 4.26).

At `-O3 -ffast-math` the compiler already inlines the function automatically, so no additional gain is observed.

3.3 Loop Fusion

After inlining, the main kernel contains two adjacent 3x3 loop nests that compute the horizontal and vertical Sobel responses over the same window. Fusing them into a single nest (Listings 1 and 2) halves the loop-control work per pixel: the `ii` and `jj` updates, comparisons, and branches.

Listing 1: Before loop fusion

```
for (ii = -1; ii <= 1; ii++) {
  for (jj = -1; jj <= 1; jj++) {
    res_horiz += input[(i+ii)*SIZE + j+jj] * horiz_operator[ii+1][jj+1];
  }
}
for (ii = -1; ii <= 1; ii++) {
  for (jj = -1; jj <= 1; jj++) {
    res_vert += input[(i+ii)*SIZE + j+jj] * vert_operator[ii+1][jj+1];
  }
}
```

Listing 2: After loop fusion

```
for (ii = -1; ii <= 1; ii++) {
  for (jj = -1; jj <= 1; jj++) {
    res_horiz += input[(i+ii)*SIZE + j+jj] * horiz_operator[ii+1][jj+1];
    res_vert  += input[(i+ii)*SIZE + j+jj] * vert_operator[ii+1][jj+1];
  }
}
```

Table 9: Execution time at `-O0`: loop fusion

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`02_sobel_function_inlining`	1124.0	2.2	1.14×
`03_sobel_loop_fusion`	1072.8	0.5	1.20×

Table 10: Execution time at `-O3 -ffast-math`: loop fusion

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`02_sobel_function_inlining`	20.6	0.1	8.46×
`03_sobel_loop_fusion`	20.6	0.1	8.44×

At `-O0` the fusion alone gives 1.05× (Table 9). At `-O3 -ffast-math` the compiler fully unrolls both 3×3 loops itself, so their loop control already disappears and fusing them gives no gain (Table 10).

Fusing the PSNR calculation into the main Sobel loop was also attempted, but proved slower under both flags: 0.94× at `-O0` (1142.6 vs. 1072.8 ms) and 0.25× at `-O3 -ffast-math` (83.4 vs. 20.6 ms). The `-O3` vectorization report explains the collapse: in the unfused version both the Sobel and PSNR loops are vectorized at 16 pixels per iteration, whereas the fused loop is not vectorized at all. This variant was therefore discarded, and PSNR fusion is revisited later in both optimization chains.

3.4 Convolution Flatten

After loop fusion, the 3×3 convolution loop of Listing 2 runs a fixed 9 iterations per pixel, so it can be removed completely by writing out all nine terms of both operators (Listing 3). This changes three things. First, the loop control disappears: the updates and comparisons of `ii` and `jj` and the loop branches. Second, the nine neighbouring pixels are read once into local variables and shared by both operators, so the loads from `input` drop from 18 to 9 per pixel. Third, the address arithmetic `(i+ii)*SIZE + j+jj` and the operator indices `[ii+1][jj+1]` are no longer computed at run time, since all offsets are now constants.

Listing 3: Flattened convolution

```
p00 = input[(i - 1)*SIZE + j - 1];
p01 = input[(i - 1)*SIZE + j];
/* ... p02 to p21 ... */
p22 = input[(i + 1)*SIZE + j + 1];

res_horiz = 0;
res_horiz += p00 * horiz_operator[0][0];
res_horiz += p01 * horiz_operator[0][1];
/* ... */
res_horiz += p22 * horiz_operator[2][2];

res_vert = 0;
res_vert += p00 * vert_operator[0][0];
/* ... */
res_vert += p22 * vert_operator[2][2];
```

Table 11: Execution time at `-O0`: convolution flatten

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`03_sobel_loop_fusion`	1072.8	0.5	1.20×
`04_sobel_conv_flatten`	679.1	2.9	1.89×

Table 12: Execution time at `-O3 -ffast-math`: convolution flatten

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`03_sobel_loop_fusion`	20.6	0.1	8.44×
`04_sobel_conv_flatten`	20.6	0.1	8.43×

At `-O0` this step alone gives 1.58×, much more than loop fusion (1.05×), because besides the loop control it also halves the pixel loads and removes the index arithmetic.

At `-O3 -ffast-math` there is no gain: clang's unroll report (`-Rpass=loop-unroll`) shows that the compiler already completely unrolls both 3-iteration convolution loops of the previous version, so the manual flattening produces the same code.

3.5 Constant Folding

In the flattened kernel the filter coefficients are still read from the global arrays `horiz_operator` and `vert_operator` for every term. Since their values are fixed, we substituted them directly into the expressions and simplified the result (Listing 4). Each Sobel operator has three zero coefficients, so six of the 18 terms disappear, including both terms of the centre pixel `p11`, which no longer needs to be loaded. The ±1 coefficients become plain additions and subtractions, and only the four terms with coefficient ±2 keep a multiplication. Per pixel, this removes the 18 coefficient loads, one of the nine pixel loads and most of the multiplications.

Listing 4: Sobel operators and the folded expressions

```
char horiz_operator[3][3] = {{-1, 0, 1},
                             {-2, 0, 2},
                             {-1, 0, 1}};
char vert_operator[3][3]  = {{ 1,  2,  1},
                             { 0,  0,  0},
                             {-1, -2, -1}};

/* After folding */
res_horiz = -p00 + p02 - 2*p10 + 2*p12 - p20 + p22;
res_vert  =  p00 + 2*p01 + p02 - p20 - 2*p21 - p22;
```

Table 13: Execution time at `-O0`: constant folding

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`04_sobel_conv_flatten`	679.1	2.9	1.89×
`05_sobel_constant_folding`	518.6	1.1	2.47×

Table 14: Execution time at `-O3 -ffast-math`: constant folding

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`04_sobel_conv_flatten`	20.6	0.1	8.43×
`05_sobel_constant_folding`	15.5	0.2	11.24×

This step gives 1.31× at `-O0` and, unlike the previous steps, also 1.33× at `-O3 -ffast-math`. The operator arrays are non-`const` global variables, so the compiler cannot assume that their values stay unchanged and keeps loading them and multiplying. Declaring the two arrays `const` in the previous version, without any manual folding, gives the same result at `-O3`: the compiler folds the coefficients itself and the time drops to the same ∼15.5 ms. At `-O0`, however, `const` has no effect, since the coefficients are still loaded and multiplied, so the manual folding is needed for the `-O0` chain.

3.6 Loop-Invariant Code Motion and Common Subexpression Elimination

Both techniques avoid recomputing the same value: loop-invariant code motion moves a computation whose inputs do not change inside a loop out of it, and common subexpression elimination computes an expression that appears several times once and reuses the result. In the kernel, the row offsets `(i-1)*SIZE`, `i*SIZE` and `(i+1)*SIZE` depend only on `i`, so they were moved out of the `j` loop and are now computed once per row instead of once per pixel. Inside the `j` loop, the index `row + j` of each of the three rows is computed once and shared by the three loads of that row and by the `output` store (Listing 5). The same LICM was applied to `i*SIZE` in the PSNR loop.

Listing 5: LICM and CSE of the index arithmetic

```
for (i = 1; i < SIZE-1; i++) {
  r0 = (i - 1)*SIZE;          /* LICM: once per row */
  r1 = i*SIZE;
  r2 = (i + 1)*SIZE;
  for (j = 1; j < SIZE-1; j++) {
    idx0 = r0 + j;            /* CSE: once per pixel */
    idx1 = r1 + j;
    idx2 = r2 + j;
    p00 = input[idx0 - 1];
    p01 = input[idx0];
    p02 = input[idx0 + 1];
    /* ... */
    output[idx1] = ...;
  }
}
```

Table 15: Execution time at `-O0`: LICM and CSE

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`05_sobel_constant_folding`	518.6	1.1	2.47×
`06_sobel_licm_cse`	508.3	0.9	2.52×

Table 16: Execution time at `-O3 -ffast-math`: LICM and CSE

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`05_sobel_constant_folding`	15.5	0.2	11.24×
`06_sobel_licm_cse`	15.0	0.3	11.56×

The gain is small at both levels. At `-O0` the step gives only 1.02×: the multiplications by `SIZE` are removed from the inner loop, but every new variable (`r0`–`r2`, `idx0`–`idx2`) is kept on the stack and has to be stored and loaded again, which cancels out most of the saving. The hardware counters confirm this: the executed instructions drop by only 2.8% (about 12 of the ∼430 instructions per pixel) and the cycles by 2.0%, while the branch count is identical.

At `-O3 -ffast-math` the compiler already performs LICM and CSE on its own, and the difference (1.03×) is within about one standard deviation, so no clear gain can be claimed.

3.7 Strength Reduction

Strength reduction replaces an expensive operation with a cheaper equivalent one. The main candidate in the kernel is `pow(x, 2)`, which is called three times per pixel: twice for the gradient magnitude and once for the PSNR. Each call goes to the general-purpose math library routine, which works on `double` values and must handle any exponent, so it costs a function call, `int` to `double` conversions and internal checks just to square a number. We replaced all three calls with a plain multiplication, and the multiplications by 2 in the gradient expressions with left shifts (Listing 6). The values stay small enough for `int`: the positive coefficients of each Sobel operator sum to 4 and the negative ones to −4, so $|g_x|, |g_y| \leq 4 \cdot 255 = 1020$ and $g_x^2 + g_y^2 \leq 2{,}080{,}800$.

Listing 6: Replacing pow with multiplication and 2* with a shift

```
/* Before */
res_horiz = -p00 + p02 - 2*p10 + 2*p12 - p20 + p22;
res_vert  =  p00 + 2*p01 + p02 - p20 - 2*p21 - p22;
p = pow(res_horiz, 2) + pow(res_vert, 2);
t = pow((output[idx] - golden[idx]), 2);
PSNR += t;

/* After */
res_horiz = -p00 + p02 - (p10 << 1) + (p12 << 1) - p20 + p22;
res_vert  =  p00 + (p01 << 1) + p02 - p20 - (p21 << 1) - p22;
p = (res_horiz * res_horiz) + (res_vert * res_vert);
diff = output[idx] - golden[idx];
PSNR += diff * diff;
```

Table 17: Execution time at `-O0`: strength reduction

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`06_sobel_licm_cse`	508.3	0.9	2.52×
`07_sobel_strength_reduction`	183.8	0.4	6.98×

Table 18: Execution time at `-O3 -ffast-math`: strength reduction

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`06_sobel_licm_cse`	15.0	0.3	11.56×
`07_sobel_strength_reduction`	12.7	0.3	13.73×

At `-O0` this is the largest single step so far (2.77×): removing the three library calls per pixel removes about two-thirds of the runtime. At `-O3 -ffast-math` the gain is smaller (1.19×), because `-ffast-math` already lets the compiler replace `pow(x, 2)` with a multiplication. However, `pow` takes and returns `double`, so the integer argument is first converted and the compiler must keep the squaring in `double` arithmetic: it cannot switch to an integer multiplication, because it does not know that the result fits in an `int`. Our version squares integers directly, so each vector register holds four values instead of two and fewer `int` to `double` conversions are needed.

The shifts contribute a small part of the `-O0` gain. At `-O0` the compiler emits a real multiplication for `2*x`. In a separate test that alternated the two variants of this version, the

shift was consistently about 1.6% faster. At `-O3 -ffast-math` the generated code is identical either way, since the compiler already performs this reduction itself. Replacing the squarings with a lookup table of squares was also tried and rejected: it gave no gain at `-O0` and made the `-O3` version 2.6× slower (39.2 vs. 14.9 ms), because the table lookups prevent the compiler from vectorizing the Sobel loop. The square root, the remaining expensive operation, is reduced differently in the `-O0` and `-O3` chains.

3.8 Integer PSNR Accumulator

After strength reduction, every squared difference of the PSNR loop is an integer, but it is still added to the `double` variable `PSNR`, so each of the 4094×4094 additions first converts the integer to `double` and then performs a floating-point addition. We accumulate the sum in a 64-bit integer instead and convert it to `double` only once, after the loop (Listing 7). The result is exactly the same: the sum is at most $4094^2 \cdot 255^2 \approx 1.1 \cdot 10^{12}$, which fits easily in an `unsigned long long`, and every partial sum is an integer that a `double` also represents exactly.

Listing 7: Integer accumulation of the squared error

```
/* Before */
diff = output[idx] - golden[idx];
PSNR += diff * diff;            /* int -> double, then FP add */

/* After */
unsigned long long mse = 0;
...
diff = output[idx] - golden[idx];
mse += diff * diff;             /* integer add */
...
PSNR = (double)mse;             /* one conversion after the loop */
```

Table 19: Execution time at `-O0`: integer PSNR accumulator

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`07_sobel_strength_reduction`	183.8	0.4	6.98×
`08_sobel_int_psnr`	152.7	0.2	8.40×

Table 20: Execution time at `-O3 -ffast-math`: integer PSNR accumulator

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`07_sobel_strength_reduction`	12.7	0.3	13.73×
`08_sobel_int_psnr`	11.5	0.4	15.11×

At `-O0` the step gives 1.20×. The addition into `PSNR` forms a chain of 16.8 M dependent operations, since each one needs the result of the previous one. At `-O0` `PSNR` is kept on the stack, so with a `double` accumulator every link of the chain is a load, a conversion to `double`, a floating-point addition and a store. With an integer accumulator the conversion disappears and the floating-point addition becomes a much cheaper integer addition, so each link of the chain becomes shorter.

At `-O3 -ffast-math` the step gives 1.10×. Here the `double` sum was already vectorized, because `-ffast-math` allows the compiler to reorder it: the vectorization report shows the PSNR loop vectorized with 16 pixels per iteration in both versions. The difference lies in the width of the data. A 128-bit vector register holds only two `double` values, so in the previous version every squared difference must be converted to `double` before it can be added. With an integer accumulator the squares are added with integer vector additions and no conversions, so each vectorized iteration does less work.

4 Optimizations for -O0

4.1 Loop Unrolling

The inner loop over `j` was unrolled 16 times, both in the Sobel kernel and in the PSNR loop. Each iteration now computes 16 consecutive output pixels of a row (Listing 8). Since $4094 = 16 \cdot 255 + 14$, the last 14 pixels of every row are handled by a remainder loop that processes one pixel at a time.

Unrolling saves work in two ways. First, the loop control (the update and comparison of `j`, the loop branch and the computation of `idx0`–`idx2`) now runs once per 16 pixels instead of once per pixel. Second, neighbouring output pixels share most of their inputs: the window of pixel j overlaps with those of pixels $j-1$ and $j+1$. In the unrolled body each of the three rows is loaded once, as 18 consecutive pixels that serve all 16 outputs, so the loads from `input` drop from 8 (after constant folding) to $3 \cdot 18/16 \approx 3.4$ per pixel.

Listing 8: Loop unrolled 16 times

```
for (j = 1; j < SIZE-16; j += 16) {
  idx0 = r0 + j;
  idx1 = r1 + j;
  idx2 = r2 + j;
  p0_0 = input[idx0 - 1];        /* 18 pixels of each row,  */
  p0_1 = input[idx0];            /* loaded once and shared  */
  /* ... */                      /* by the 16 outputs       */
  p2_17 = input[idx2 + 16];

  res_horiz0_0 = -p0_0 + p0_2 - (p1_0 << 1) + (p1_2 << 1) - p2_0 + p2_2;
  res_vert0_0  =  p0_0 + (p0_1 << 1) + p0_2 - p2_0 - (p2_1 << 1) - p2_2;
  /* ... the same for the other 15 pixels ... */
}
for (; j < SIZE-1; j++) {        /* remainder: last 14 pixels */
  /* ... one pixel per iteration ... */
}
```

Table 21: Execution time at `-O0`: loop unrolling

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`08_sobel_int_psnr`	152.7	0.2	8.40×
`09_sobel_unroll`	86.6	0.2	14.82×

The step gives 1.76×, the largest gain of the `-O0` chain. At `-O0` every variable lives on the stack, so each load from `input` that is saved also saves the store and reload of the corresponding local variable, and each loop iteration that disappears saves the loads, stores and branch of the loop control. The cost is a much larger loop body, with 54 pixel variables and 16 sets of gradient variables. This is not a problem here: the code of the whole program is about 7 KB, far below the 192 KB L1 instruction cache of a performance core.

4.2 Memory Reads

The previous step reuses loaded pixels only horizontally, between neighbouring pixels of the same row. Vertically there is the same overlap: the window of a pixel in row i shares two of its three rows with the pixel directly below it, in row $i + 1$. To reuse loads in both directions, the loop over `i` is also unrolled: each iteration now computes a block of r rows by k columns instead of one row of 16 pixels.

A block of $r \times k$ output pixels needs the input pixels of the block plus a one-pixel border around it, i.e. $(r + 2) \times (k + 2)$ loads. The number of loads per output pixel is therefore

$$\frac{(r+2)(k+2)}{r\,k} = 1 + \frac{2(r+k)+4}{r\,k}.$$

For a fixed number of pixels per iteration rk, only the sum $r + k$ changes, so the loads are smallest when r and k are as close as possible, i.e. for a square block. Table 22 shows this for 16 pixels per iteration: the 1×16 row of the previous step needs 54 loads, while a 4×4 block needs only 36 for the same 16 output pixels. We therefore use a 4×4 block (Listing 9). Since $4094 = 4 \cdot 1023 + 2$, the last two rows and the last two columns of the image are handled by remainder loops.

Table 22: Loads from `input` for 16 output pixels per iteration

Block ($r \times k$)	**Loads per iteration**	**Loads per pixel**	$r + k$
1×16 (previous step)	$3 \cdot 18 = 54$	3.38	17
2×8	$4 \cdot 10 = 40$	2.50	10
4×4 (this step)	$6 \cdot 6 = 36$	2.25	8

Listing 9: 4×4 block

```
for (i = 1; i < SIZE-4; i += 4) {      /* 4 rows per iteration    */
  /* r0 ... r5: offsets of the 6 input rows (i-1) ... (i+4)       */
  for (j = 1; j < SIZE-4; j += 4) {    /* 4 columns per iteration */
    /* idx0 ... idx5 = r0 + j ... r5 + j                          */
    p0_0 = input[idx0 - 1];            /* 6 x 6 pixels, each      */
    /* ... */                         /* loaded once and shared  */
    p5_5 = input[idx5 + 4];            /* by the 16 outputs       */
    /* ... 16 output pixels ... */
  }
  for (; j < SIZE-1; j++) { ... }      /* remainder: last 2 columns */
}
for (; i < SIZE-1; i++) { ... }        /* remainder: last 2 rows    */
```

Table 23: Execution time at `-O0`: memory reads

Executable	**Mean (ms)**	**Std. dev. (ms)**	**Speedup**
`09_sobel_unroll`	86.6	0.2	14.82×
`10_sobel_mem_reads`	80.9	0.1	15.85×

The step gives 1.07×. The gain is smaller than the one-third reduction in loads because the loads are only part of the work per pixel: the gradient arithmetic, the square root and the output store are the same as before, and at `-O0` each of these also goes through the stack.

4.3 Square-Root Lookup Table

After the previous steps, the most expensive remaining operation per pixel is `(int)sqrt(p)`: the integer $p = g_x^2 + g_y^2$ is converted to `double`, its square root is computed, the result is converted back to an integer and then compared with 255 for clipping. Since p is an integer between 0 and $2 \cdot 1020^2 = 2{,}080{,}800$, all possible results can be computed in advance and stored in a table `sqrt_table[p]` $= \min(\lfloor\sqrt{p}\rfloor, 255)$. Each pixel then needs a single byte load instead of the square root, the two conversions and the clipping branch (Listing 10).

The table has one byte per possible p, about 2 MB. It is filled without computing any square root: all values of p from i^2 to $(i+1)^2 - 1$ have the same integer square root i, so the table consists of 255 runs of equal values, followed by 255 for every $p \geq 255^2$. The table is filled at the start of the timed region, so its cost is included in the measurements, and the output is identical to the previous version.

Listing 10: Square-root lookup table

```
#define MAX_P (2 * 1020 * 1020)        /* largest gx^2 + gy^2 */
/* sqrt_table[p] = min((int)sqrt(p), 255) for every p = gx^2 + gy^2 */
unsigned char sqrt_table[MAX_P + 1];

for (i = 0; i < 255; i++)                  /* p in [i^2, (i+1)^2 - 1] */
  memset(&sqrt_table[i*i], i, 2*i + 1);    /* has (int)sqrt(p) = i    */
memset(&sqrt_table[255*255], 255, MAX_P + 1 - 255*255);

/* Before */
res = (int)sqrt(p);
if (res > 255) output[idx1] = 255;
else           output[idx1] = (unsigned char)res;

/* After */
output[idx1] = sqrt_table[p];
```

Table 24: Execution time at `-O0`: square-root lookup table

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`10_sobel_mem_reads`	80.9	0.1	15.85×
`11_sobel_lut`	66.9	0.1	19.18×

The step gives 1.21×. In most pixels the gradient magnitude is below 255, so the clipping branch of the previous version was rarely taken and was predicted correctly almost every time. The gain therefore comes mainly from removing the square root and the two conversions, not from removing the branch.

4.4 PSNR Fusion

Until now the PSNR was computed in a separate loop that runs over the whole image a second time, after the Sobel loop has written all of `output`. In this step the two loops are fused: right after a 4×4 block of output pixels is computed and stored, the squared differences of the same 16 pixels from `golden` are added to the error sum (Listing 11). The second loop over the image disappears completely, together with its loop control and its index computations. The 16 differences reuse the indices `idx1`–`idx4` that the Sobel stores have already computed, and each output value is read back right after it was written, while it is still in the L1 cache.

Listing 11: PSNR fused into the 4×4 block

```
/* ... the 16 output pixels of the block are stored ... */
d0_0 = output[idx1]     - golden[idx1];
d0_1 = output[idx1 + 1] - golden[idx1 + 1];
/* ... */
d3_3 = output[idx4 + 3] - golden[idx4 + 3];
mse += d0_0 * d0_0 + d0_1 * d0_1 + /* ... */ + d3_3 * d3_3;
```

Table 25: Execution time at `-O0`: PSNR fusion

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`11_sobel_lut`	66.9	0.1	19.18×
`12_sobel_psnr_fusion`	62.7	0.05	20.44×

The step gives 1.07×. The same fusion did not help when it was tried in the common path, on a version without unrolling. In the 4×4 block, however, the PSNR work shares the indices and the loop control of the Sobel computation, so the fusion removes work that the separate loop had to repeat.

5 Optimizations for `-O3 -ffast-math`

5.1 Single-Precision Square Root

At `-O3 -ffast-math` the Sobel loop of the last common version is already vectorized, and the most expensive operation left in it is the square root. It is computed in `double`, so the integer p is converted to a 64-bit value and a 128-bit vector register holds only two of them. We compute it in single precision instead, with `sqrtf` (Listing 12). A `float` is half as wide, so each vector register holds four values and every vector instruction of the square root and the conversions processes twice as many pixels.

Listing 12: Single-precision square root

```
/* Before */
res = (int)sqrt(p);

/* After */
res = (int)sqrtf((float)p);
```

The result is exactly the same. Every possible p is at most 2,080,800 $< 2^{24}$, so it is represented exactly as a `float`, and a check of all values from 0 to 2,080,800 showed that `(int)sqrtf((float)p)` and `(int)sqrt(p)` are equal for every one of them.

Table 26: Execution time at `-O3 -ffast-math`: single-precision square root

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`08_sobel_int_psnr`	11.5	0.4	15.11×
`09_sobel_sqrtf`	8.8	0.1	19.83×

The step gives 1.31×. This is the `-O3` counterpart of the lookup table of the `-O0` chain: both remove the cost of the double-precision square root, but a table lookup cannot be vectorized, while the single-precision square root keeps the loop vectorized and doubles the number of pixels per instruction.

5.2 Compiler Hints

The last step gives the compiler information that it cannot infer by itself (Listing 13). The three image pointers are declared `restrict`, which promises that the memory each of them points to is accessed only through that pointer, so `output` never overlaps `input` or `golden`. The loop counters, row offsets and indices are declared `register`. The third common hint, `const`, was already examined in the constant folding step: it lets the compiler fold the operator coefficients itself, but after that step the coefficient arrays are no longer used in the kernel.

Listing 13: Compiler hints

```
double sobel(unsigned char *restrict input, unsigned char *restrict output,
             unsigned char *restrict golden)
{
  register int i, j, r, r0, r1, r2, idx, idx0, idx1, idx2;
  register unsigned long long mse = 0;
  /* ... */
```

Table 27: Execution time at `-O3 -ffast-math`: compiler hints

Executable	Mean (ms)	Std. dev. (ms)	Speedup
`09_sobel_sqrtf`	8.8	0.1	19.83×
`10_sobel_compiler_hints`	8.7	0.1	20.00×

The gain is marginal, about 1%, which is within one standard deviation. The `register` keyword has no effect, since at `-O3` the compiler allocates registers on its own. For `restrict`, the vectorization report is identical with and without the hints: both versions vectorize the same loops with the same width. Without `restrict`, a compiler that vectorizes a loop over pointers that might overlap typically adds a run-time check before the loop, which tests whether the arrays overlap and falls back to a scalar version if they do. `restrict` makes this check unnecessary. The check runs only before each loop and not for every pixel, so removing it saves very little, which matches the measured result. At `-O0` the same hints produce exactly the same machine code as without them (Table 28).

6 Conclusion

6.1 Rejected Optimizations

Besides the steps of the two chains, several transformations were tried and rejected (Table 28). Each one is compared with the version it would have replaced or extended, shown as the base.

Table 28: Rejected optimizations

Transformation	Base	Base (ms)	Time (ms)	Speedup
`-O0` chain				
Compiler hints	`12_sobel_psnr_fusion`	62.7	63.3	0.99×
`-O3 -ffast-math` chain				
Loop unrolling (1×16)	`08_sobel_int_psnr`	11.5	48.6	0.24×
Square-root lookup table	`08_sobel_int_psnr`	11.5	22.7	0.51×
PSNR fusion	`09_sobel_sqrtf`	8.8	8.7	1.00×

In the `-O0` chain, the compiler hints have no effect: the compiler generates exactly the same code with and without them.

In the `-O3 -ffast-math` chain, the transformations that helped at `-O0` are harmful because they get in the way of the vectorizer. After manual unrolling by 16, the vectorization report shows the Sobel loop vectorized with only 4 pixels per iteration instead of 16, and the version becomes more than four times slower. With the lookup table the Sobel loop is not vectorized at all, because each pixel reads the table at a different position, and the time doubles. Finally, fusing the PSNR into the Sobel loop gives the same time as before: the fused loop is still vectorized, but with 8 instead of 16 pixels per iteration.

6.2 Final Results

Table 29 summarizes all versions that were kept. The common steps reduce the execution time by 8.40× at `-O0` and by 15.11× at `-O3 -ffast-math`. The two chains then bring the final speedup over the baseline to 20.44× at `-O0` and 20.00× at `-O3 -ffast-math`. At `-O0` most of the gain comes from manual work that the compiler does not do itself, such as inlining, unrolling and strength reduction. At `-O3` the compiler already performs most of these transformations, and the manual changes that helped were those that gave it information it could not infer: the order of the loops, the constant coefficients, the integer types and the precision of the square root.

Table 29: Execution time and speedup over the baseline of all kept versions

	`-O0`		`-O3 -ffast-math`	
Executable	**ms**	**Speedup**	**ms**	**Speedup**
`00_sobel_orig`	1282.5	1.00×	173.9	1.00×
`01_sobel_loop_interchange`	1252.9	1.02×	20.6	8.46×
`02_sobel_function_inlining`	1124.0	1.14×	20.6	8.46×
`03_sobel_loop_fusion`	1072.8	1.20×	20.6	8.44×
`04_sobel_conv_flatten`	679.1	1.89×	20.6	8.43×
`05_sobel_constant_folding`	518.6	2.47×	15.5	11.24×
`06_sobel_licm_cse`	508.3	2.52×	15.0	11.56×
`07_sobel_strength_reduction`	183.8	6.98×	12.7	13.73×
`08_sobel_int_psnr`	152.7	8.40×	11.5	15.11×
`-O0` chain				
`09_sobel_unroll`	86.6	14.82×	–	–
`10_sobel_mem_reads`	80.9	15.85×	–	–
`11_sobel_lut`	66.9	19.18×	–	–
`12_sobel_psnr_fusion`	62.7	20.44×	–	–
`-O3 -ffast-math` chain				
`09_sobel_sqrtf`	–	–	8.8	19.83×
`10_sobel_compiler_hints`	–	–	8.7	20.00×

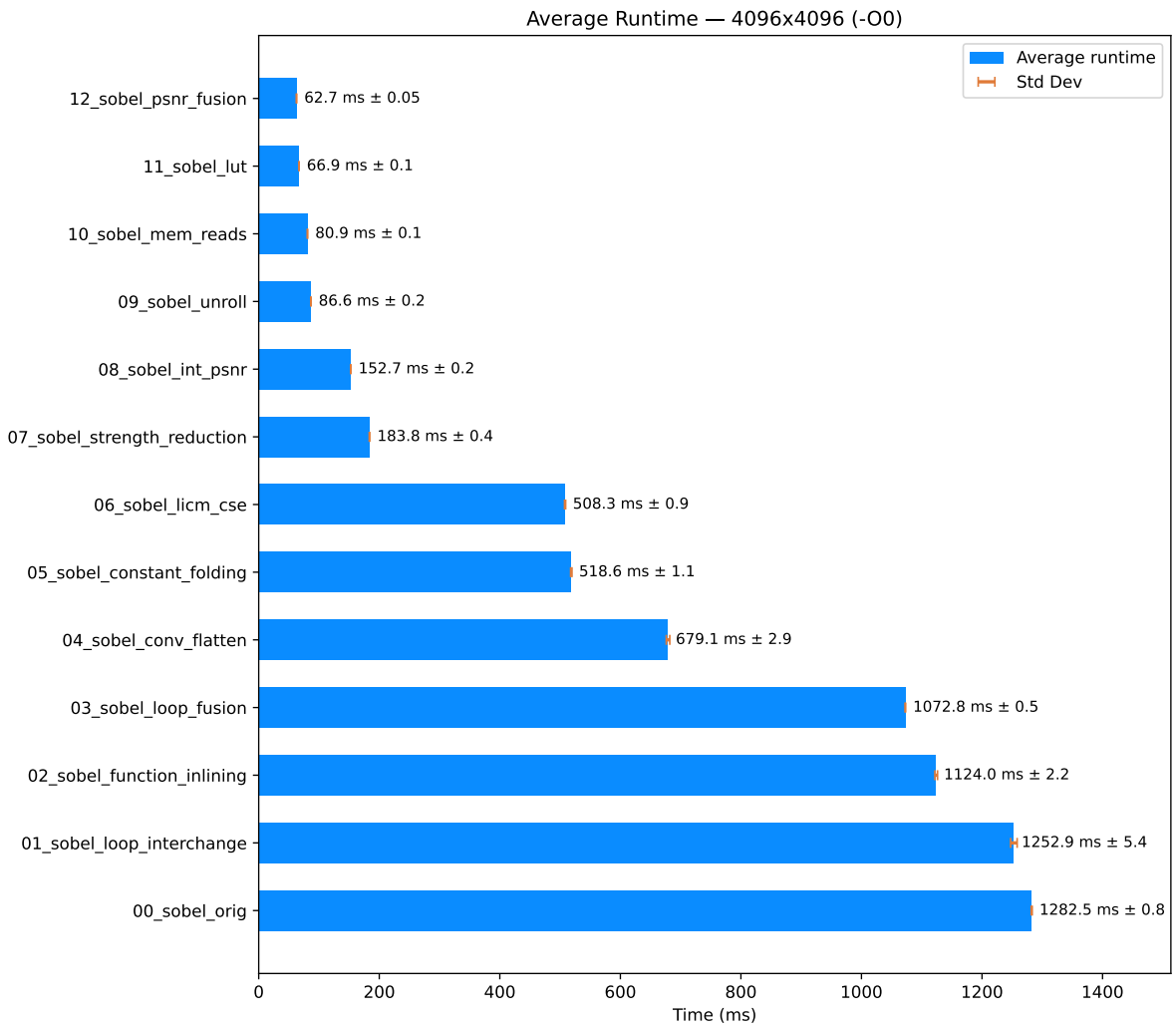


Figure 1: Execution time of all kept versions at `-O0`

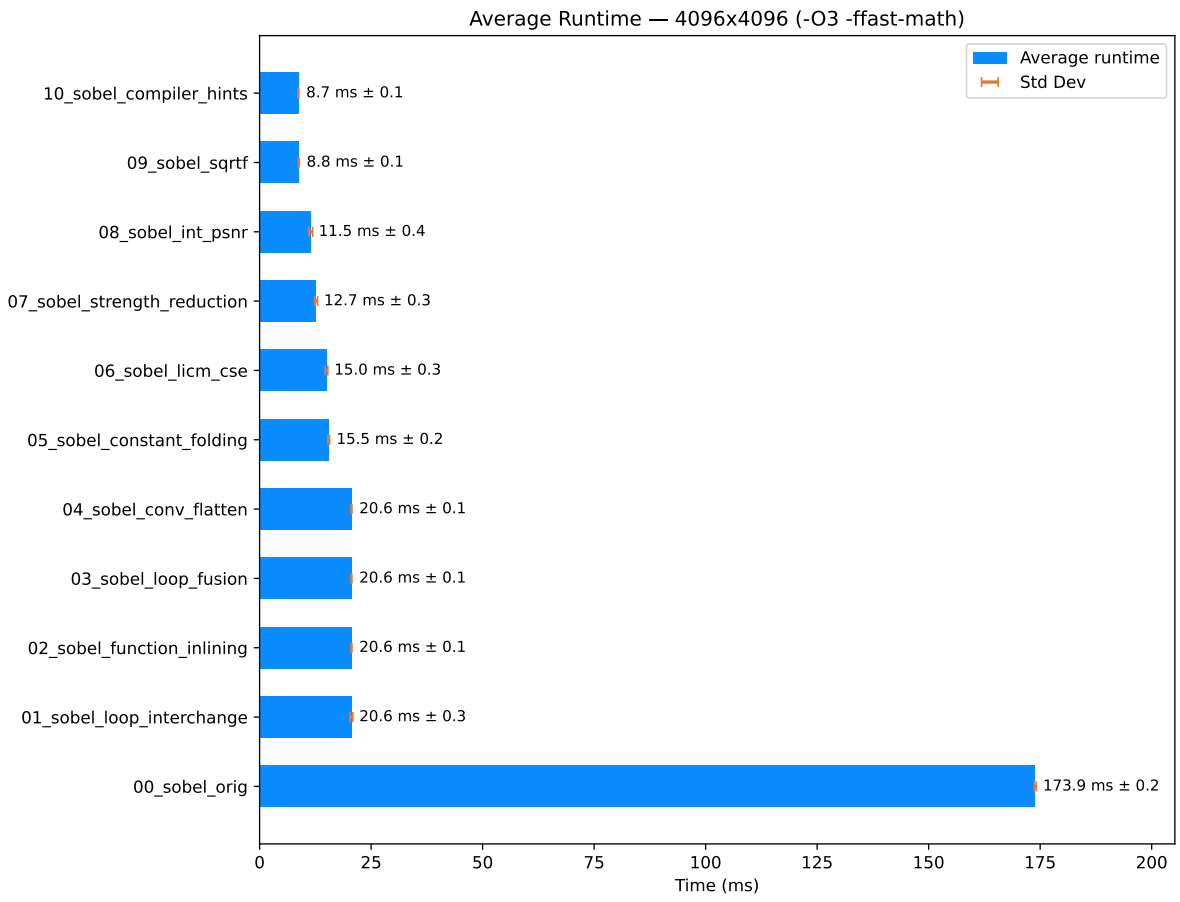


Figure 2: Execution time of all kept versions at `-O3 -ffast-math`